МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №2

по дисциплине «Интеллектуальные методы анализа данных»

Тема: Реализация метода классификации данных с использованием природных временных рядов

Студент гр. 3381 _____ Щеглов М.А.

Преподаватель _____ Мандрикова Б.С.

Санкт-Петербург
2026

Цель работы.

Построение классификатора данных геофизического мониторинга на примере наблюдений нейтронных мониторов.

Задание.

Вариант 14: Алма-Ата (AATB), 2–10 мая 2017 г.

1. Подготовить данные. Скачать данные из источника [https://www.nmdb.eu/nest/] за периоды и станции, указанные по вариантам в таблицах. Использовать данные с одноминутным временным разрешением. При наличии пропусков в данных заполнить их медианным значением (функция median в MatLab).

2. Сформировать векторы-истории. Временной ряд разбить на последовательные блоки (векторы) длиной m, каждый из которых должен содержать m прошлых значений. Последний вектор в ряду будет «тестовой» точкой для поиска соседей. Величину m выбрать равной 180 отсчетов.

3. Найти ближайших соседей. Для «тестового» вектора вычислить расстояние (метрику) до всех остальных векторов-историй (использовать 3 различные метрики). Число соседей (k) определить равным 2–4.

4. Использовать соседей для прогноза. На основе найденных ближайших соседей построить предсказания для будущих значений ряда. При нахождении нескольких схожих отрезков нужно взять простое среднее следующих за ними значений.

5. Сделать соответствующие выводы, оформить отчет по работе.

Выполнение работы.

Загрузка данных. Данные нейтронного монитора Алма-Ата B (AATB) получены из базы NMDB через интерфейс NEST (https://www.nmdb.eu/nest/) за период с 02.05.2017 00:00 по 10.05.2017 23:59 UTC. Использовались одноминутные значения скорости счёта, скорректированные на атмосферное давление и эффективность детектора.

Обработка данных. Сервер NMDB не передаёт строки за те минуты, когда данных нет, поэтому ряд сначала приводится к равномерной сетке с шагом 1 мин (метод reindex), и отсутствующие отсчёты становятся пропусками. Вместо 12960 значений получено 12922, то есть пропущено 38 отсчётов (0,29 %). Пропуски короткие, по 2–5 мин: несколько перерывов утром 2 мая и ежесуточный перерыв 00:29–00:31 начиная с 5 мая. Тот же перерыв попал и в интервал фактических значений 11 мая.

Кроме пропусков, в ряду найдены выбросы по правилу трёх сигм (значения вне интервала $\mu \pm 3\sigma$). Большая часть из 29 выбросов одиночные значения на краях распределения шума, а 8 мая в 08:55–11:17 UTC видна серия из восьми аппаратных всплесков от 1481,8 до 1685,4 имп./с (рис. 1, а). Пропуски и выбросы заменены медианой значений, оставшихся после их исключения . Так же обработан файл фактческих значений. Итоги обработки приведены в таблице 1.

Таблица 1 — Результаты предварительной обработки данных

Показатель	**Исторические данные (02.05–10.05.2017)**	**Фактические значения (11.05.2017, 00:00–00:59)**
Ожидаемое число отсчётов	12960	60
Получено отсчётов	12922	57
Пропусков	38 (0,29 %)	3 (5,00 %)
Среднее μ, имп./с	1422,60	1418,19
СКО σ, имп./с	13,06	12,16
Границы нормы μ ± 3σ, имп./с	[1383,44; 1461,77]	[1381,72; 1454,66]
Выбросов	29 (0,22 %)	1 (1,67 %)
Медиана для замены, имп./с	1422,50	1419,30

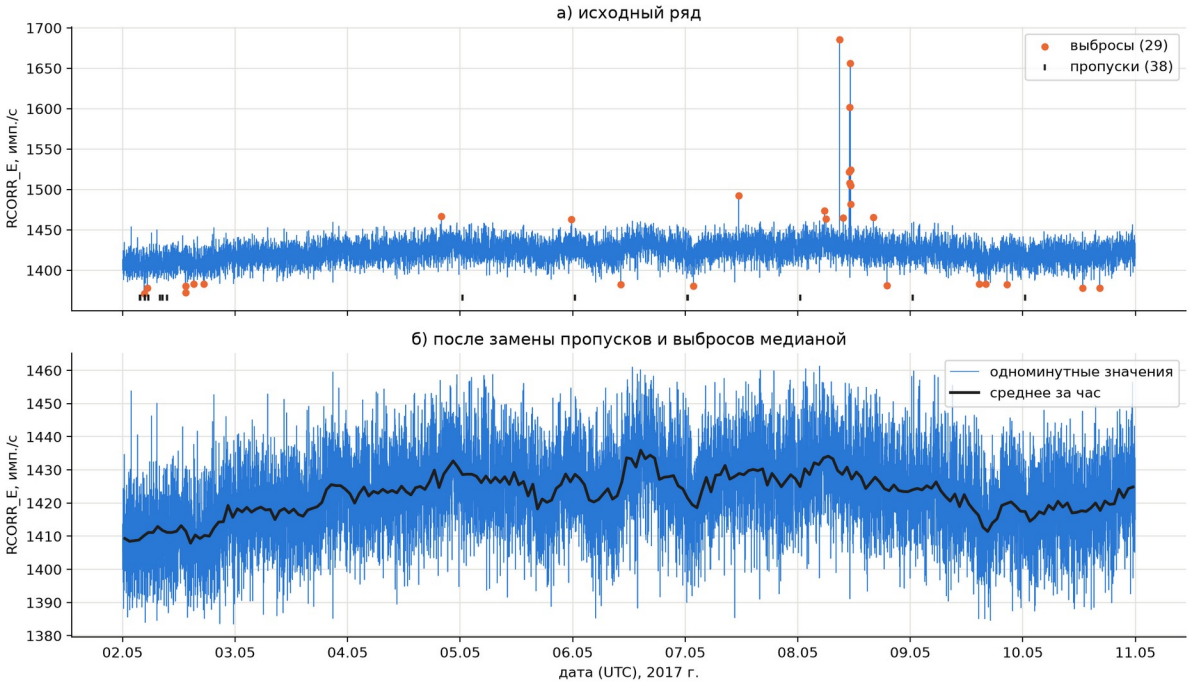


Рис. 1 - Исходный (а) и обработанный (б) ряд AATB, 2–10 мая 2017 г.

Среднечасовые значения обработанного ряда меняются медленно: от 1408 имп./с (02.05) до 1436 имп./с (06.05), то есть всего примерно на 2%. Одноминутные значения колеблются вокруг среднечасового уровня со среднеквадратическим отклонением около 10,7 имп./с, так что на минутном масштабе ряд в основном состоит из случайных флуктуаций счёта.

Формирование векторов историй. Длина обработанного ряда N = 12960. Тестовым вектором служат последние m = 180 значений ряда (10.05.2017 21:00–23:59). Векторы истории - все отрезки длины 180, которые

получаются скользящим окном с шагом в один отсчёт и целиком лежат до тестового вектора, то есть не пересекаются с ним. Начальный индекс такого отрезка i = 0, 1, …, N − 2m, всего получен 12601 вектор-история (матрица 12601 × 180). Для каждого вектора запоминаются 60 следующих за ним значений, по ним и строится прогноз.

Поиск ближайших соседей. Для тестового вектора x и каждого вектора-истории y вычислены три расстояния: евклидово (1), манхэттенское (2) и расстояние Чебышева (3):

$$d_E(x, y) = \sqrt{\left(\Sigma_{j=1..m} (x_j - y_j)^2 \right)} \quad (1)$$

$$d_M(x, y) = \Sigma_{j=1..m} |x_j - y_j| \quad (2)$$

$$d_ч(x, y) = max_{j=1..m} |x_j - y_j| \quad (3)$$

Расстояния считаются сразу для всей матрицы векторов историй. По каждой метрике векторы-истории упорядочены по возрастанию расстояния и отобраны 4 ближайших; при k = 2 и k = 3 используются первые 2 и 3 из них. Найденные соседи приведены в таблице 2.

Таблица 2 — Ближайшие соседи тестового вектора

Метрика	**№**	**Индекс**	**Интервал (UTC)**	**Расстояние**
Евклидова	1	9737	08.05 18:17–21:16	172,168
	2	3350	04.05 07:50–10:49	174,489
	3	3138	04.05 04:18–07:17	178,199
	4	9725	08.05 18:05–21:04	180,199
Манхэттенская	1	1728	03.05 04:48–07:47	1867,800
	2	9737	08.05 18:17–21:16	1887,200
	3	3350	04.05 07:50–10:49	1897,200
	4	9799	08.05 19:19–22:18	1900,200

Метрика	№	Индекс	Интервал (UTC)	Расстояние
Чебышева	1	9734	08.05 18:14–21:13	32,000
	2	5366	05.05 17:26–20:25	32,500
	3	5067	05.05 12:27–15:26	32,800
	4	9766	08.05 18:46–21:45	32,800

Все три метрики нашли соседей в интервале 08.05 18:05–22:18 UTC: средний уровень счёта в нём (1424,0 имп./с) близок к среднему тестового вектора (1423,6 имп./с). Евклидова и манхэттенская метрики дали два общих соседа. Некоторые соседи сдвинуты друг относительно друга всего на 12-32 отсчёта (9725 и 9737, 9734 и 9766), то есть почти перекрываются, и следующие за ними значения во многом совпадают.

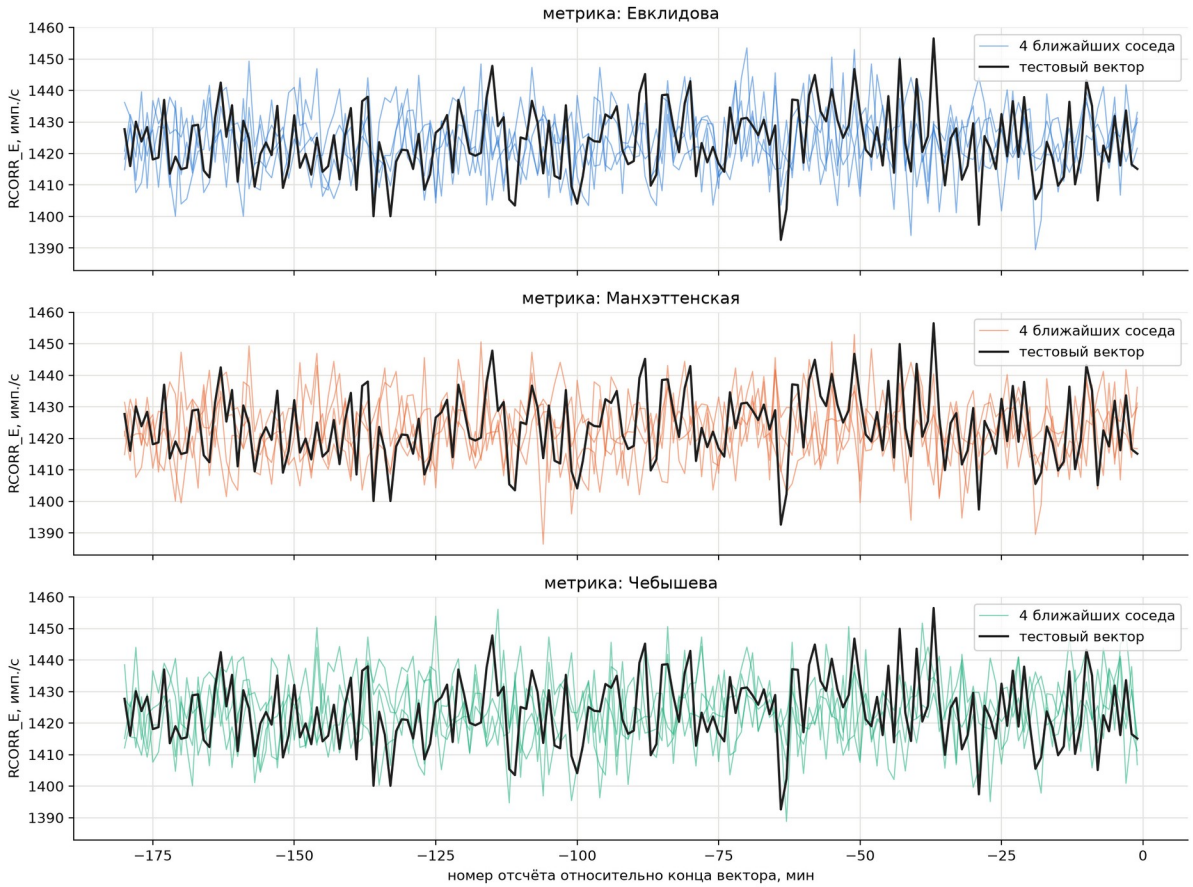


Рис. 2 — Тестовый вектор и 4 ближайших соседа по каждой метрике

Из рис. 2 видно, что соседи не повторяют форму тестового вектора по точкам: совпадают только средний уровень и размах колебаний. Это подтверждает расчёт: по всем 12601 векторам историям коэффициент корреляции расстояния с модулем разности средних уровней тестового вектора и вектора-истории равен 0,86 для евклидовой, 0,85 для манхэттенской и 0,39 для метрики Чебышева. Иначе говоря, евклидова и манхэттенская метрики в основном отбирают отрезки с тем же средним уровнем, а расстояние Чебышева определяется одним наибольшим отклонением, то есть по сути случайным всплеском шума.

Прогноз по соседям. Прогноз для заданного k - простое среднее значений, следующих за k ближайшими соседями (функция knn_forecast):

$$\hat{y}_t = (1/k) \cdot \Sigma_{s=1..k}\, x(i_s + m + t), \quad t = 0, 1, \ldots, 59 \qquad (4)$$

где x(·) значение ряда, i_1, …, i_k - индексы соседей, t - номер прогнозируемого шага. Для сравнения взят простейший (наивный) прогноз среднее значение тестового вектора, 1423,62 имп./с, постоянное на всём горизонте. Прогнозы по каждой метрике для k = 2, 3, 4 показаны на рис. 3–5, общий вид — на рис. 6.

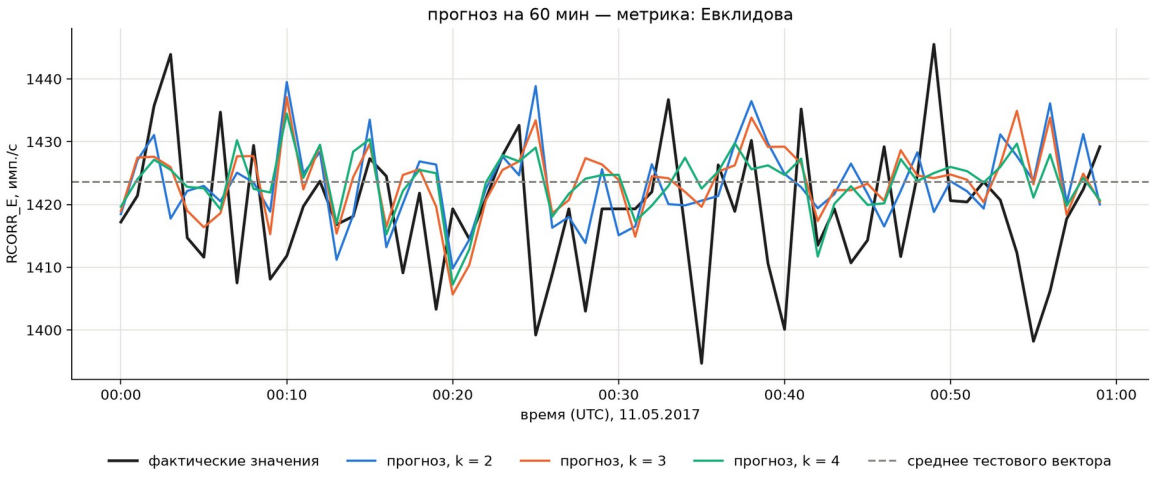


Рис. 3 - Прогноз по евклидовой метрике

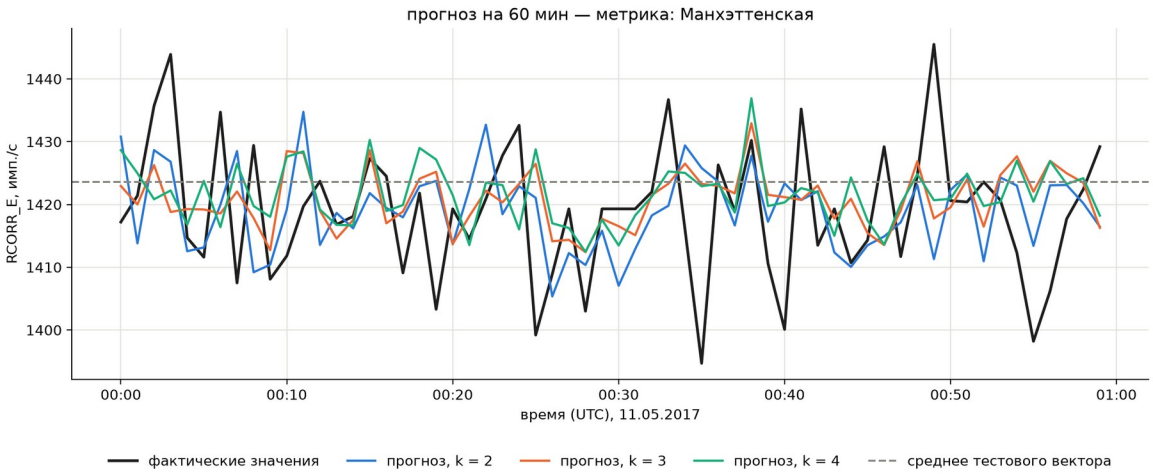


Рис. 4 - Прогноз по манхэттенской метрике

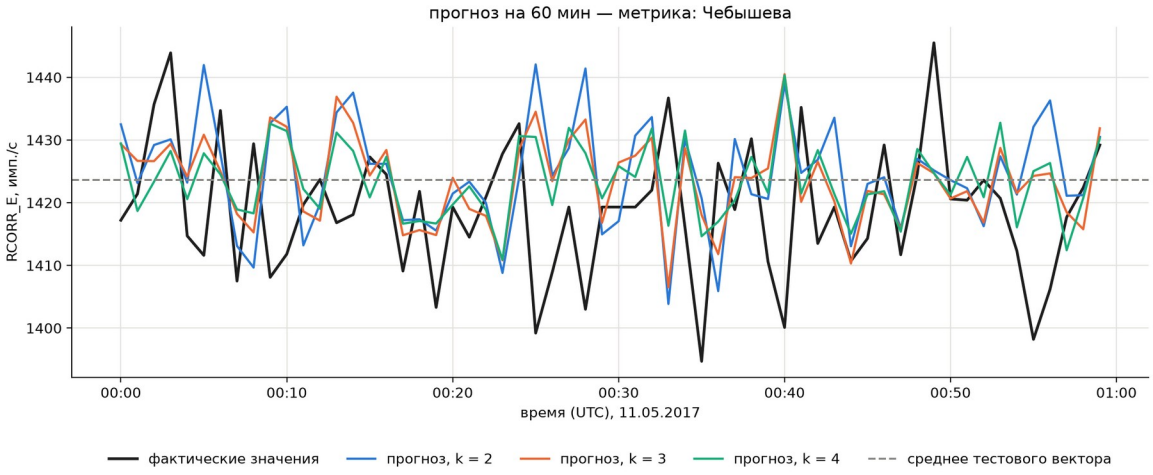


Рис. 5 - Прогноз по метрике Чебышева

С ростом k прогноз становится более гладким: при усреднении k отрезков случайные колебания частично взаимно гасятся. Отдельные пики и провалы фактического ряда (например, 1394,7 имп./с в 00:35 и 1445,5 имп./с в 00:49) прогноз не воспроизводит, а его собственные пики не совпадают с фактическими по времени. Сильнее всего колеблется прогноз по метрике Чебышева при k = 2.



Рис. 6 —-Тестовый вектор, фактические значения и прогнозы при k = 4

Оценка качества прогноза. Качество прогноза оценивалось по трём показателям: средней абсолютной ошибке MAE (5), корню из среднеквадратической ошибки RMSE (6) и средней абсолютной процентной ошибке MAPE (7), где y_t фактическое значение, n = 60:

$$MAE = (1/n) \cdot \Sigma_{t=1..n} |y_t - \hat{y}_t| \quad (5)$$

$$RMSE = \sqrt{((1/n) \cdot \Sigma_{t=1..n} (y_t - \hat{y}_t)^2)} \quad (6)$$

$$MAPE = (100\ \%/n) \cdot \Sigma_{t=1..n} |y_t - \hat{y}_t| / y_t \quad (7)$$

Результаты приведены в таблице 3.

Таблица 3 - Ошибки прогноза на 60 мин

Метрика	**k**	**MAE, имп./с**	**RMSE, имп./с**	**MAPE, %**
Евклидова	2	10,363	13,524	0,732
	3	9,664	12,887	0,683
	4	9,635	12,414	0,681
Манхэттенская	2	9,429	11,994	0,665
	3	**8,994**	**11,755**	**0,635**
	4	9,235	12,002	0,652
Чебышева	2	12,471	16,332	0,881
	3	11,199	14,326	0,791
	4	10,327	13,225	0,729
Среднее тестового вектора	—	9,211	11,603	0,651

Лучший результат дала манхэттенская метрика при k = 3, худший метрика Чебышева при k = 2 . Наивный прогноз даёт MAE = 9,211 имп./с. Лучше него оказался только вариант манхэттенская метрика, k = 3 (на 2,4 %), вариант с k = 4 практически ему равен , остальные хуже. СКО фактических значений равно 10,60 имп./с, то есть ошибка всех прогнозов того же порядка, что и разброс самого ряда.

Чтобы проверить влияние числа соседей, программа дополнительно вычисляет MAE для k от 1 до 20 (рис. 7). Ошибка уменьшается с ростом k для всех метрик. Для евклидовой и манхэттенской метрик при k ≥ 5 MAE ниже, чем у наивного прогноза (8,26–9,13 имп./с), а для метрики Чебышева ошибка приближается к наивному прогнозу (9,27 имп./с) только при k = 15–20 и не становится меньше. Чем больше соседей, тем ближе прогноз к среднему уровню похожих отрезков, то есть метод фактически оценивает ожидаемый уровень счёта в следующий час, а не форму ряда.

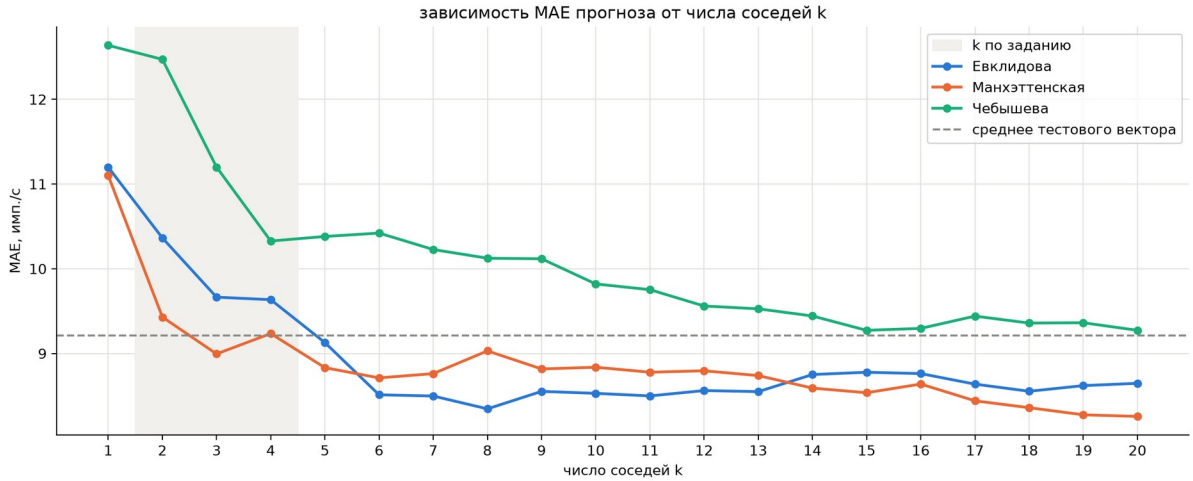


Рис. 7 - Зависимость MAE прогноза от числа соседей k

Все оценки получены по одному тестовому вектору, поэтому разница в несколько десятых имп./с между вариантами статистически ненадёжна: для строгого сравнения расчёт нужно повторить для многих тестовых векторов.

Выводы.

В ходе работы построен прогноз одноминутного ряда нейтронного монитора Алма-Ата B за 2–10 мая 2017 г. методом ближайших соседей на

векторах-историях длины m = 180. Метод работает как классификатор: тестовый вектор относится к группе наиболее похожих отрезков истории, а прогноз строится по тому, что следовало за ними.

При подготовке данных 38 пропусков (0,29 %) и 29 выбросов (0,22 %), в том числе серия аппаратных всплесков 8 мая, заменены медианой 1422,5 имп./с.. По евклидовой, манхэттенской метрикам и метрике Чебышева найдены ближайшие соседи и построены прогнозы на 60 мин при k = 2, 3, 4.

Ошибка прогноза MAE составила от 8,99 до 12,47 имп./с при СКО фактических значений 10,60 имп./с и MAE наивного прогноза (среднее тестового вектора) 9,21 имп./с. Лучший вариант - манхэттенская метрика при k = 3. Хуже всего работает метрика Чебышева: она определяется одной наибольшей разностью, то есть случайным выбросом шума, и поэтому хуже отбирает соседей. Увеличение k уменьшает ошибку за счёт усреднения: в пределах k = 2-4 для евклидовой метрики и метрики Чебышева лучший результат даёт k = 4, для манхэттенской k = 3.

Основная причина низкой точности - свойства самих данных: на минутном масштабе ряд состоит в основном из случайных флуктуаций счёта, а медленные вариации за девять дней составляют лишь около 2 % (среднечасовые значения от 1408 до 1436 имп./с). Поэтому отдельные будущие значения по прошлым 180 отсчётам предсказать нельзя: евклидова и манхэттенская метрики фактически отбирают отрезки с близким средним уровнем, прогноз повторяет шум соседей и при больших k сводится к оценке среднего уровня.

ПРИЛОЖЕНИЕ А. ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: imad2.py

```
import json
import re
import sys
import urllib.parse
import urllib.request
from pathlib import Path


import matplotlib


if "--show" not in sys.argv:
    matplotlib.use("Agg")


import matplotlib.dates as mdates
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from numpy.lib.stride_tricks import sliding_window_view


# ---------------------------------------------------------------------------
# параметры работы
# ---------------------------------------------------------------------------
STATION = "AATB"                        # Алма-Ата B
HISTORY_START = "2017-05-02 00:00"      # период варианта
HISTORY_END = "2017-05-10 23:59"
ACTUAL_START = "2017-05-11 00:00"       # фактические значения после периода,
ACTUAL_END = "2017-05-11 00:59"         # с которыми сравнивается прогноз


M = 180                    # длина вектора-истории, отсчётов
HORIZON = 60               # число прогнозируемых значений
K_VALUES = (2, 3, 4)       # число ближайших соседей
K_SCAN = range(1, 21)      # диапазон k для дополнительного анализа
SIGMA = 3                  # порог выбросов (правило трёх сигм)


BASE_DIR = Path(__file__).resolve().parent
DATA_DIR = BASE_DIR / "data"
FIG_DIR = BASE_DIR / "figures"
HISTORY_FILE = DATA_DIR / "aatb_history.html"
ACTUAL_FILE = DATA_DIR / "aatb_actual.html"
RESULTS_FILE = BASE_DIR / "results.json"


METRIC_NAMES = {
    "euclid": "Евклидова",
    "manhattan": "Манхэттенская",
    "chebyshev": "Чебышева",
}


# цвета графиков: фактические данные — тёмные, ряды — в фиксированном порядке
COLOR_ACTUAL = "#1f1f1f"
COLOR_MUTED = "#8a8985"
SERIES_COLORS = ["#2a78d6", "#eb6834", "#1baf7a"]



# ---------------------------------------------------------------------------
# загрузка данных
# ---------------------------------------------------------------------------
def nmdb_url(station, start, end):
    """Формирует запрос к NEST (nmdb.eu) на одноминутные данные RCORR_E."""
    t0, t1 = pd.Timestamp(start), pd.Timestamp(end)
```

```
    params = {
        "formchk": 1, "stations[]": station, "tabchoice": "revori",
        "dtype": "corr_for_efficiency", "tresolution": 0, "yunits": 0,
        "date_choice": "bydate", "output": "ascii",
        "start_day": t0.day, "start_month": t0.month, "start_year": t0.year,
        "start_hour": t0.hour, "start_min": t0.minute,
        "end_day": t1.day, "end_month": t1.month, "end_year": t1.year,
        "end_hour": t1.hour, "end_min": t1.minute,
    }
    return "https://www.nmdb.eu/nest/draw_graph.php?" + urllib.parse.urlencode(params)



def download_if_missing(path, station, start, end):
    """Скачивает файл с сайта NMDB, если его ещё нет на диске."""
    if path.exists():
        return
    path.parent.mkdir(parents=True, exist_ok=True)
    url = nmdb_url(station, start, end)
    print(f"скачивание {path.name} с {url}")
    with urllib.request.urlopen(url, timeout=120) as response:
        path.write_bytes(response.read())



def parse_nmdb(path):
    """Читает HTML-ответ NEST и возвращает ряд значений с индексом-временем."""
    html = path.read_text(encoding="utf-8")
    block = re.search(r"<pre><code>(.*?)</code></pre>", html, re.DOTALL)
    text = block.group(1) if block else html


    times, values = [], []
    for line in text.splitlines():
        line = line.strip()
        if not re.match(r"^\d{4}-\d{2}-\d{2} \d{2}:\d{2}", line):
            continue
        stamp, value = line.split(";")[:2]
        times.append(pd.Timestamp(stamp))
        values.append(pd.to_numeric(value.strip(), errors="coerce"))


    if not times:
        raise ValueError(f"в файле {path} не найдены строки с данными")
    return pd.Series(values, index=pd.DatetimeIndex(times), name="RCORR_E")



# ---------------------------------------------------------------------------
# подготовка данных
# ---------------------------------------------------------------------------
def prepare(raw, start, end, title):
    """
    Приводит ряд к равномерной минутной сетке, находит пропуски и выбросы
    (правило трёх сигм) и заменяет их медианой «чистых» значений.
    """
    grid = pd.date_range(start, end, freq="min")
    series = raw.reindex(grid)


    missing = series.isna()
    mean, std = series.mean(), series.std()
    lower, upper = mean - SIGMA * std, mean + SIGMA * std
    outliers = (series < lower) | (series > upper)


    median = series[~missing & ~outliers].median()
    processed = series.copy()
    processed[missing | outliers] = median


    info = {
        "title": title,
```

```
        "expected": len(grid),
        "received": int(raw.notna().sum()),
        "missing": int(missing.sum()),
        "missing_share": float(missing.mean() * 100),
        "mean": float(mean),
        "std": float(std),
        "lower": float(lower),
        "upper": float(upper),
        "outliers": int(outliers.sum()),
        "outliers_share": float(outliers.mean() * 100),
        "outlier_max": float(series[outliers].max()) if outliers.any() else None,
        "median": float(median),
    }


    print(f"--- {title} ---")
    print(f"ожидалось отсчётов: {info['expected']}, получено: {info['received']}")
    print(f"пропусков: {info['missing']} ({info['missing_share']:.2f}%)")
    print(f"границы нормы ({SIGMA} сигмы): [{lower:.2f}; {upper:.2f}]")
    print(f"выбросов: {info['outliers']} ({info['outliers_share']:.2f}%)")
    print(f"медиана для замены: {median:.2f}\n")
    return processed, series, missing, outliers, info



# ---------------------------------------------------------------------------
# метод ближайших соседей
# ---------------------------------------------------------------------------
def build_history_vectors(series, m, horizon):
    """
    Тестовый вектор — последние m значений ряда. Векторы-истории — все отрезки
    длины m (окно со сдвигом 1), которые целиком лежат до тестового вектора;
    для каждого запоминаются следующие за ним horizon значений.
    """
    n = len(series)
    test = series[n - m:]
    count = n - 2 * m + 1
    histories = sliding_window_view(series, m)[:count]
    futures = sliding_window_view(series, horizon)[m:m + count]
    return test, histories, futures



def distances(histories, test):
    """Расстояния от тестового вектора до каждого вектора-истории."""
    diff = np.abs(histories - test)
    return {
        "euclid": np.sqrt(np.sum(diff ** 2, axis=1)),
        "manhattan": np.sum(diff, axis=1),
        "chebyshev": np.max(diff, axis=1),
    }



def knn_forecast(futures, dist, k):
    """Прогноз — простое среднее значений, следующих за k ближайшими соседями."""
    idx = np.argsort(dist, kind="stable")[:k]
    return futures[idx].mean(axis=0), idx



def errors(actual, predicted):
    diff = actual - predicted
    return {
        "mae": float(np.mean(np.abs(diff))),
        "rmse": float(np.sqrt(np.mean(diff ** 2))),
        "mape": float(np.mean(np.abs(diff) / actual) * 100),
    }
```

```
# ---------------------------------------------------------------------------
# графики
# ---------------------------------------------------------------------------
def style_axes(ax, ylabel="RCORR_E, имп./с"):
    ax.set_ylabel(ylabel)
    ax.grid(True, color="#e4e3df", linewidth=0.8)
    ax.set_axisbelow(True)
    for side in ("top", "right"):
        ax.spines[side].set_visible(False)



def save(fig, name):
    fig.tight_layout()
    fig.savefig(FIG_DIR / name, dpi=200)
    print(f"сохранён график: figures/{name}")



def plot_series(raw, processed, missing, outliers):
    fig, axes = plt.subplots(2, 1, figsize=(12, 7), sharex=True)


    axes[0].plot(raw.index, raw.values, color=SERIES_COLORS[0], linewidth=0.6)
    axes[0].scatter(raw.index[outliers], raw[outliers], s=18,
                    color=SERIES_COLORS[1], zorder=3,
                    label=f"выбросы ({int(outliers.sum())})")
    gap_times = raw.index[missing]
    axes[0].scatter(gap_times, np.full(len(gap_times), raw.min() - 5), s=18,
                    marker="|", color=COLOR_ACTUAL, zorder=3,
                    label=f"пропуски ({int(missing.sum())})")
    axes[0].set_title("а) исходный ряд")
    axes[0].legend(loc="upper right")
    style_axes(axes[0])


    hourly = processed.resample("60min").mean()
    axes[1].plot(processed.index, processed.values, color=SERIES_COLORS[0],
                 linewidth=0.6, label="одноминутные значения")
    axes[1].plot(hourly.index + pd.Timedelta(minutes=30), hourly.values,
                 color=COLOR_ACTUAL, linewidth=2, label="среднее за час")
    axes[1].set_title("б) после замены пропусков и выбросов медианой")
    axes[1].legend(loc="upper right")
    axes[1].xaxis.set_major_formatter(mdates.DateFormatter("%d.%m"))
    axes[1].set_xlabel("дата (UTC), 2017 г.")
    style_axes(axes[1])
    save(fig, "fig1_series.png")



def plot_neighbors(test, histories, neighbors, k):
    fig, axes = plt.subplots(3, 1, figsize=(12, 9), sharex=True, sharey=True)
    steps = np.arange(-M, 0)
    for ax, (key, name), color in zip(axes, METRIC_NAMES.items(), SERIES_COLORS):
        idx = neighbors[key][:k]
        for j, i in enumerate(idx):
            ax.plot(steps, histories[i], color=color, linewidth=0.8, alpha=0.55,
                    label=f"{k} ближайших соседа" if j == 0 else None)
        ax.plot(steps, test, color=COLOR_ACTUAL, linewidth=1.6,
                label="тестовый вектор")
        ax.set_title(f"метрика: {name}")
        ax.legend(loc="upper right")
        style_axes(ax)
    axes[-1].set_xlabel("номер отсчёта относительно конца вектора, мин")
    save(fig, "fig2_neighbors.png")



def plot_forecast(times, actual, baseline, preds, key, number):
    fig, ax = plt.subplots(figsize=(12, 5))
    ax.plot(times, actual, color=COLOR_ACTUAL, linewidth=2,
```

```
            label="фактические значения")
    for k, color in zip(K_VALUES, SERIES_COLORS):
        ax.plot(times, preds[key][k], color=color, linewidth=1.6,
                label=f"прогноз, k = {k}")
    ax.axhline(baseline, color=COLOR_MUTED, linewidth=1.4, linestyle="--",
               label="среднее тестового вектора")
    ax.set_title(f"прогноз на {HORIZON} мин — метрика: {METRIC_NAMES[key]}")
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%H:%M"))
    ax.set_xlabel("время (UTC), 11.05.2017")
    ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.18), ncol=5,
              frameon=False)
    style_axes(ax)
    save(fig, f"fig{number}_forecast_{key}.png")



def plot_overview(test_times, test, times, actual, preds, k):
    fig, ax = plt.subplots(figsize=(12, 5))
    ax.plot(test_times, test, color=COLOR_MUTED, linewidth=0.9,
            label="тестовый вектор (последние 180 значений)")
    ax.plot(times, actual, color=COLOR_ACTUAL, linewidth=1.6,
            label="фактические значения")
    for (key, name), color in zip(METRIC_NAMES.items(), SERIES_COLORS):
        ax.plot(times, preds[key][k], color=color, linewidth=1.6,
                label=f"прогноз — {name}, k = {k}")
    ax.axvline(times[0], color=COLOR_ACTUAL, linewidth=1, linestyle=":")
    ax.set_title("тестовый вектор и прогноз следующих 60 значений")
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%H:%M"))
    ax.set_xlabel("время (UTC), 10–11.05.2017")
    ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.18), ncol=3,
              frameon=False)
    style_axes(ax)
    save(fig, "fig6_overview.png")



def plot_k_scan(scan, baseline_mae):
    fig, ax = plt.subplots(figsize=(12, 5))
    ks = list(K_SCAN)
    ax.axvspan(min(K_VALUES) - 0.5, max(K_VALUES) + 0.5, color="#f1f0ec",
               label="k по заданию")
    for (key, name), color in zip(METRIC_NAMES.items(), SERIES_COLORS):
        ax.plot(ks, scan[key], color=color, linewidth=2, marker="o",
                markersize=5, label=name)
    ax.axhline(baseline_mae, color=COLOR_MUTED, linewidth=1.4, linestyle="--",
               label="среднее тестового вектора")
    ax.set_title("зависимость MAE прогноза от числа соседей k")
    ax.set_xlabel("число соседей k")
    ax.set_xticks(ks)
    ax.legend(loc="upper right")
    style_axes(ax, "MAE, имп./с")
    save(fig, "fig7_k_scan.png")



# ---------------------------------------------------------------------------
# основная программа
# ---------------------------------------------------------------------------
def main():
    FIG_DIR.mkdir(exist_ok=True)
    download_if_missing(HISTORY_FILE, STATION, HISTORY_START, HISTORY_END)
    download_if_missing(ACTUAL_FILE, STATION, ACTUAL_START, ACTUAL_END)


    # 1. подготовка данных
    hist_raw = parse_nmdb(HISTORY_FILE)
    act_raw = parse_nmdb(ACTUAL_FILE)
    print("первые строки исторических данных:")
    print(hist_raw.head(), "\n")
```

```
hist, hist_grid, hist_missing, hist_outliers, hist_info = prepare(
    hist_raw, HISTORY_START, HISTORY_END, "исторические данные")
act, _, _, _, act_info = prepare(
    act_raw, ACTUAL_START, ACTUAL_END, "фактические значения")


series = hist.to_numpy(dtype=float)
actual = act.to_numpy(dtype=float)
assert len(actual) == HORIZON, "число фактических значений не равно горизонту"


# 2. векторы-истории
test, histories, futures = build_history_vectors(series, M, HORIZON)
test_times = hist.index[-M:]
print(f"длина ряда N = {len(series)}, m = {M}, горизонт = {HORIZON}")
print(f"векторов-историй: {len(histories)}, матрица {histories.shape}, "
      f"будущие значения {futures.shape}\n")


# 3. ближайшие соседи по трём метрикам
dists = distances(histories, test)
neighbors = {key: np.argsort(d, kind="stable")[:max(K_VALUES)]
             for key, d in dists.items()}
neighbor_rows = []
for key, name in METRIC_NAMES.items():
    print(f"метрика: {name}")
    for rank, i in enumerate(neighbors[key], 1):
        t0 = hist.index[i]
        t1 = hist.index[i + M - 1]
        print(f"  сосед {rank}: индекс = {i}, "
              f"{t0:%d.%m %H:%M}–{t1:%d.%m %H:%M}, "
              f"расстояние = {dists[key][i]:.3f}")
        neighbor_rows.append({
            "metric": key, "rank": rank, "index": int(i),
            "start": f"{t0:%d.%m %H:%M}", "end": f"{t1:%d.%m %H:%M}",
            "distance": float(dists[key][i]),
        })
    print()


# насколько расстояние определяется разницей средних уровней векторов
level_diff = np.abs(histories.mean(axis=1) - test.mean())
level_corr = {key: float(np.corrcoef(d, level_diff)[0, 1])
              for key, d in dists.items()}
print("корреляция расстояния с |разностью средних уровней|:")
for key, name in METRIC_NAMES.items():
    print(f"  {name}: {level_corr[key]:.3f}")
print()


# 4. прогноз по соседям и оценка качества
baseline = float(test.mean())
baseline_err = errors(actual, np.full(HORIZON, baseline))
preds, quality = {}, []
for key in METRIC_NAMES:
    preds[key] = {}
    for k in K_VALUES:
        pred, _ = knn_forecast(futures, dists[key], k)
        preds[key][k] = pred
        quality.append({"metric": key, "k": k, **errors(actual, pred)})


print("качество прогноза (MAE / RMSE / MAPE):")
for row in quality:
    print(f"  {METRIC_NAMES[row['metric']]:<14} k = {row['k']}: "
          f"MAE = {row['mae']:.3f}, RMSE = {row['rmse']:.3f}, "
          f"MAPE = {row['mape']:.3f}%")
print(f"  среднее тестового вектора ({baseline:.2f}): "
      f"MAE = {baseline_err['mae']:.3f}, RMSE = {baseline_err['rmse']:.3f}, "
      f"MAPE = {baseline_err['mape']:.3f}%")
print(f"СКО фактических значений: {actual.std():.3f}\n")


# дополнительно: как меняется ошибка при увеличении числа соседей
```

```
    scan = {key: [errors(actual, knn_forecast(futures, dists[key], k)[0])["mae"]
                  for k in K_SCAN]
            for key in METRIC_NAMES}

    # 5. графики
    plot_series(hist_grid, hist, hist_missing, hist_outliers)
    plot_neighbors(test, histories, neighbors, max(K_VALUES))
    for number, key in enumerate(METRIC_NAMES, start=3):
        plot_forecast(act.index, actual, baseline, preds, key, number)
    plot_overview(test_times, test, act.index, actual, preds, max(K_VALUES))
    plot_k_scan(scan, baseline_err["mae"])


    results = {
        "params": {"station": STATION, "m": M, "horizon": HORIZON,
                   "k_values": list(K_VALUES), "sigma": SIGMA},
        "history": hist_info,
        "actual": act_info,
        "n_series": len(series),
        "n_vectors": len(histories),
        "test_start": f"{test_times[0]:%d.%m.%Y %H:%M}",
        "test_end": f"{test_times[-1]:%d.%m.%Y %H:%M}",
        "test_mean": baseline,
        "actual_std": float(actual.std()),
        "neighbors": neighbor_rows,
        "level_corr": level_corr,
        "quality": quality,
        "baseline": baseline_err,
        "k_scan": {key: dict(zip(K_SCAN, v)) for key, v in scan.items()},
    }
    RESULTS_FILE.write_text(json.dumps(results, ensure_ascii=False, indent=2),
                            encoding="utf-8")
    print(f"\nрезультаты сохранены в {RESULTS_FILE.name}")


    if "--show" in sys.argv:
        plt.show()



if __name__ == "__main__":
    main()
```